



Quick Reference Card v2.5

Freque Plugin Controls - Quick Reference Card v2.5

The definitive patterns for all Freque plugin controls



Window Resize Handling

For Game Plugins / Positioned Elements

```
onResize(width, height) {  
  // 1. Update positioning  
  this.player.x = width / 2;  
  this.centerX = width / 2;  
  
  // 2. Recreate elements (preserve state!)  
  this.recreateEnemies();  
  this.recreateObstacles();  
  
  // 3. Update boundaries  
  this.leftBoundary = 50;  
  this.rightBoundary = width - 50;  
}  
  
// Helper method that preserves game state  
recreateEnemies() {  
  const savedScore = this.score;  
  const savedWave = this.wave;  
  
  this.createEnemies();  
  
  this.score = savedScore;  
  this.wave = savedWave;  
}
```

✗ Common Mistakes

```
// ✗ DON'T: Forget to recreate elements
onResize(width, height) {
    // Canvas resizes but elements at old positions!
}
```

```
// ✗ DON'T: Reset game state
onResize(width, height) {
    this.createEnemies(); // Resets score/wave!
}
```

```
// ✔ DO: Recreate AND preserve state
onResize(width, height) {
    this.recreateEnemies(); // Uses helper method
}
```



Control Types



Dropdown

```
this.addControl('name', {
  type: 'dropdown',
  label: 'Label',
  className: 'dropdown-selector-mixer', // REQUIRED
  options: [
    { value: 0, label: 'Option 1' },
    { value: 1, label: 'Option 2' }
  ],
  value: 0,
  onChange: (value) => {
    this.property = value;
  }
});
```



Toggle (ON/OFF)

```
this.addControl('name', {
  type: 'checkbox',
  label: 'Label',
  checked: false,
  className: 'btn-primary-mixer', // REQUIRED for consistent styling
  onChange: (value) => {
    this.property = value;
  }
});
```

✓ Dial (Rotary Control)

```
this.addControl('name', {
  type: 'dial',          // Use dial instead of slider!
  label: 'Label',
  min: 0,
  max: 100,
  step: 1,
  value: 50,
  unit: '',              // Optional: 'x', '%', etc.
  onChange: (value) => {
    this.property = value;
  }
});
```

✓ Custom Dial Fill Color (Plugin Config)

```
// In constructor - applies to ALL dials in plugin
super('myplugin', visualizer, {
  version: '1.0.0',
  dialFillColor: '--accent-color' // Optional: CSS variable for dial fill
});
```

Available CSS Variables: - `--accent-color` - Theme accent color (recommended)

- `--highlight-color` - Theme highlight color - `--error-color` - Error color - `--success-color` - Success color - Any other CSS variable in your theme

Note: Dial outline (stroke) remains unchanged. Only fill color is customized.

✓ Mode Button

```
this.addControl('name', {
  type: 'button',
  label: 'Mode A',
  className: 'btn-toggle',
  onClick: () => {
    this.toggleMode();
  }
});

toggleMode() {
  this.mode = this.mode === 'A' ? 'B' : 'A';
  const btn = document.querySelector('[data-control="name"]');
  btn.textContent = `Mode ${this.mode}`;
}
```

✓ Action Button

```
this.addControl('name', {
  type: 'button',
  label: 'Do Something',
  onClick: () => {
    this.doSomething();
  }
});
```

Color Morphing

Basic Color Morph

```

constructor(visualizer) {
    super('myplugin', visualizer, {...});

    this.colorMorph = false;
    this.hue = 0;
    this.morphSpeed = 1.0;
}

setupControls() {
    this.addControl('colorMorph', {
        type: 'checkbox',
        label: 'Color Morph',
        checked: false,
        className: 'btn-primary-mixer',
        onChange: (value) => {
            this.colorMorph = value;
        }
    });
}

onUpdate(deltaTime, timestamp) {
    if (this.colorMorph) {
        this.hue += this.morphSpeed * deltaTime * 0.05;
        if (this.hue > 360) this.hue -= 360;
    }
}

onRender() {
    this.ctx.fillStyle = `hsl(${this.hue}, 100%, 50%)`;
    // ... render
}

```


Palette Cycling

```
constructor(visualizer) {
  super('myplugin', visualizer, {...});

  this.colorPalette = [
    { h: 0, s: 100, l: 50 },    // Red
    { h: 120, s: 100, l: 50 }, // Green
    { h: 240, s: 100, l: 50 }  // Blue
  ];
  this.paletteIndex = 0;
  this.morphProgress = 0;
}

onUpdate(deltaTime) {
  if (this.colorMorph) {
    this.morphProgress += deltaTime * 0.001;
    if (this.morphProgress >= 1.0) {
      this.morphProgress = 0;
      this.paletteIndex = (this.paletteIndex + 1) % this.colorPalette.length;
    }
  }
}

getCurrentColor() {
  const current = this.colorPalette[this.paletteIndex];
  const next = this.colorPalette[(this.paletteIndex + 1) % this.colorPalette.length];

  return {
    h: this.lerp(current.h, next.h, this.morphProgress),
    s: this.lerp(current.s, next.s, this.morphProgress),
    l: this.lerp(current.l, next.l, this.morphProgress)
  };
}
```

Audio-Reactive Morph

```
onUpdate(deltaTime, timestamp, sharedAudioData) {  
  if (this.colorMorph && sharedAudioData) {  
    const bass = sharedAudioData.bass || 0;  
  
    // Bass affects morph speed  
    const speed = this.morphSpeed * (1 + bass * 2);  
    this.hue += speed * deltaTime * 0.05;  
    if (this.hue > 360) this.hue -= 360;  
  }  
}
```

Common Mistakes

WRONG: Button for Toggle

```
// DON'T DO THIS  
this.addControl('toggle', {  
  type: 'button',          //  Wrong type  
  onClick: () => {  
    this.toggleSomething();  
  }  
});
```

✓ CORRECT: Checkbox for Toggle

```
// DO THIS
this.addControl('toggle', {
  type: 'checkbox',      // ✓ Right type
  checked: false,
  onChange: (value) => {
    this.something = value;
  }
});
```

Decision Table

Control Type	Use For	Example
checkbox	ON/OFF toggles	Audio Reactive, Color Morph
button	Mode switching	Direct ↔ Reflection
button	One-time actions	Randomize, Reset
dial	Numeric values	Size, Speed, Opacity
dropdown	Multiple options	Color Scheme, Preset

Audio Integration

Use MAL's sharedAudioData

```
onUpdate(deltaTime, timestamp, sharedAudioData) {
  if (this.audioReactive && sharedAudioData) {
    // Use provided data
    const energy = sharedAudioData.energy;
    const frequencies = sharedAudioData.frequencies;

    // Process frequency bands
    this.frequencyBands = this.getFrequencyBands(frequencies);
  }
}

getFrequencyBands(frequencies) {
  if (!frequencies) return { bass: 0, mid: 0, treble: 0 };

  const length = frequencies.length;
  return {
    bass: this.avgRange(frequencies, 0, Math.floor(length * 0.1)),
    mid: this.avgRange(frequencies, Math.floor(length * 0.1), Math.floor(length * 0.5)),
    treble: this.avgRange(frequencies, Math.floor(length * 0.5), length)
  };
}
```

DON'T Create Custom Analyser

```
// DON'T DO THIS
this.analyser = audioContext.createAnalyser(); //  No!
```

Complete Example

```

class MyPlugin extends FrequePluginBase {
  constructor(visualizer) {
    super('myplugin', visualizer, {
      version: '1.0.0',
      author: 'Your Name',
      description: 'My Plugin'
    });

    this.setupControls();
  }

  setupControls() {
    // Dropdown
    this.addControl('colorScheme', {
      type: 'dropdown',
      label: 'Color Scheme',
      className: 'dropdown-selector-mixer',
      options: [
        { value: 0, label: 'Rainbow' },
        { value: 1, label: 'Warm' }
      ],
      value: 0,
      onChange: (value) => {
        this.colorScheme = value;
      }
    });

    // Toggle
    this.addControl('audioReactive', {
      type: 'checkbox',
      label: 'Audio Reactive',
      checked: true,
      className: 'btn-primary-mixer',
      onChange: (value) => {
        this.audioReactive = value;
      }
    });
  }
}

```

```

    }
  });

  // Dial
  this.addControl('size', {
    type: 'dial',
    label: 'Size',
    min: 1,
    max: 10,
    step: 0.1,
    value: 5,
    onChange: (value) => {
      this.size = value;
    }
  });
}

onUpdate(deltaTime, timestamp, sharedAudioData) {
  if (this.audioReactive && sharedAudioData) {
    const energy = sharedAudioData.energy;
    this.size = this.size * (1 + energy);
  }
}
}

```

Checklist

Before submitting your plugin:

- ☐ All toggles use `type: 'checkbox'` with `className: 'btn-primary-mixer'`
- ☐ All numeric controls use `type: 'dial'` (not slider)
- ☐ All dropdowns have `className: 'dropdown-selector-mixer'`
- ☐

Audio uses `sharedAudioData` from MAL

- ☐ No custom analyser created
 - ☐ Buttons only for modes/actions
 - ☐ All controls have `onChange/onClick`
 - ☐ No manual DOM manipulation for toggles
 - ☐ **Window resize handling implemented (if plugin has positioned elements)**
 - ☐ **Game state preserved during resize (score, lives, wave)**
 - ☐ **Elements recreated for new canvas size on resize**
-

Quick Reference v2.5 - Use These Patterns! 

Freque Plugin Development Documentation

© 2025 Freque - Professional Audio Visualization Platform